Gradient Boosting: the Intuition

Contents

1. Overview . . . 2
2. Boosting passes mistakes forward . . . 3
3. The toy data: five students . . . 4
4. Stage 1: the mean . . . 4
5. Pseudo-residuals: the mistakes of the current model . . . 5
6. Stage 2: a tree that predicts the mistakes . . . 6
7. Adding the models together . . . 7
8. The learning rate: small steps in the right direction . . . 7
9. Stage 3: the residuals of the combined model . . . 8
10. Predicting for a new student . . . 10
11. Watching it fit a curve . . . 11
12. Learning rate against number of trees . . . 13
13. Gradient boosting compared with AdaBoost . . . 14
14. Summary . . . 16
15. Sources . . . 16
16. Key terms . . . 17

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Learning rate; Gradient descent; Log loss (binary cross entropy); Sigmoid function; Regression trees; Boosting.
- **Leads to:** XGBoost.
- **Compare with:** AdaBoost.

1. Overview

Key point: Gradient boosting starts with the mean as its first guess. Then, again and again, it trains a small **regression tree** (G-1654) on the current mistakes (the residuals) and adds a small part of that tree to the model.

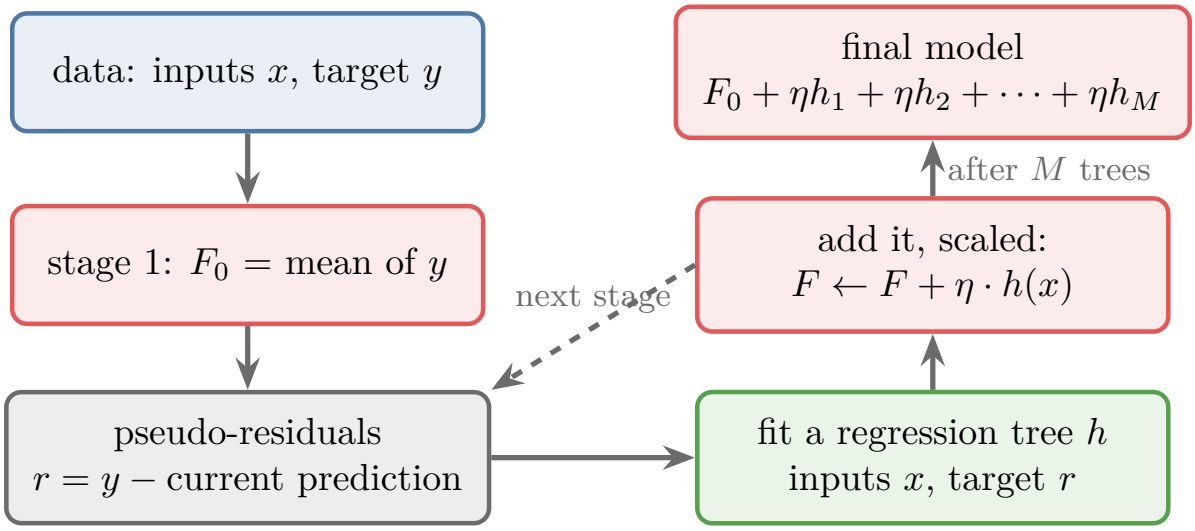


Figure 1: Gradient boosting for regression: a mean, then trees trained on residuals, each added with the learning rate

Gradient boosting (G-859) is a boosting algorithm, like **AdaBoost** (G-167): it builds a strong model by adding many weak ones (here small trees) one at a time. Gradient boosting is one of the strongest algorithms for tables of data (Grinsztajn et al. 2022), and its optimised version, XGBoost, has won many Kaggle competitions: 17 of the 29 winning solutions on Kaggle's blog in 2015 used it (Chen and Guestrin 2016). Figure 1 shows the whole method for regression. The present Note:

- follows it by hand on five students (sections 3 to 10);
- watches it fit a curve (sections 11 and 12);

- compares it with AdaBoost (section 13).

The maths behind each step is in **additive modelling** (G-174); classification is in one algorithm, a different loss.

2. Boosting passes mistakes forward

> **Key point:** Every boosting algorithm adds small models one at a time; each new model is told about the mistakes of the models before it. AdaBoost tells it through **observation** (G-1374) weights; gradient boosting tells it through the residuals.

Boosting (see a stage-wise additive model) builds a big model from small models, added one after another: a **stage-wise additive model** (G-1867). What makes it boosting is that each new model learns from the mistakes of the ones before.

Gradient boosting (G-859) changes three things in **AdaBoost** (G-167):

- **The first model.** AdaBoost starts with a **decision stump** (G-559), a tree with one split. Gradient boosting starts with a single leaf: one number, predicted for every observation (section 4).
- **The size of the trees.** AdaBoost adds stumps. Gradient boosting adds trees that are larger than a stump but still limited in size, commonly 8 to 32 leaves.
- **How the trees are scaled.** AdaBoost gives each stump its own say. Gradient boosting multiplies every tree by the same number, the **learning rate** (G-1067) (section 8).

Section 13 returns to these differences with data. The fourth difference is the main one. The two algorithms pass on the mistakes in different ways:

- **AdaBoost** raises the weights of the observations it got wrong and draws a new dataset by weight, so the next stump focuses on those observations (see update the observation weights).
- **Gradient boosting** writes the mistakes down as numbers, one per observation, and trains the next model to predict exactly those numbers.

Figure 2 puts the two side by side. Watch the middle box: the only difference is the form the mistakes take before the next model sees them.

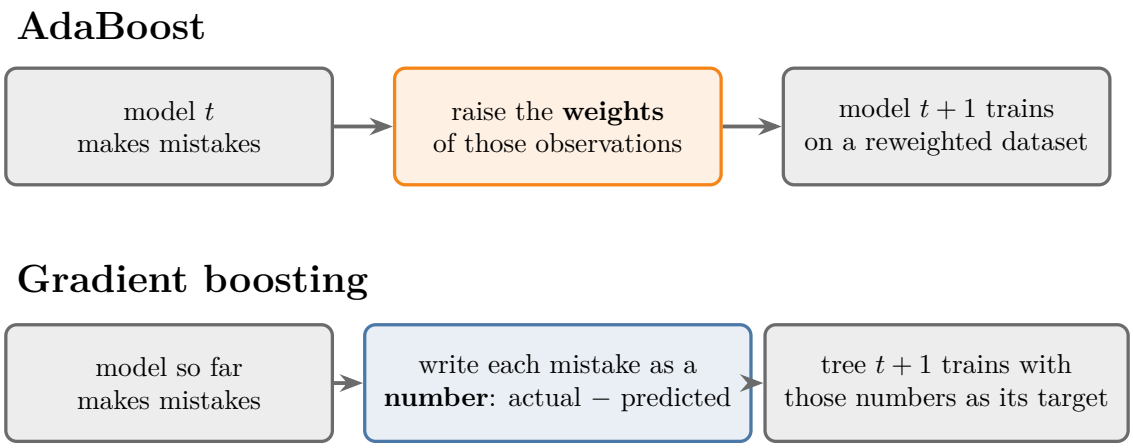


Figure 2: AdaBoost passes mistakes on as observation weights; gradient boosting passes them on as numbers that become the next tree's target

3. The toy data: five students

> **Key point:** Two features (IQ and CGPA), one numeric target (package in LPA): a regression problem.

Each student is an **observation** (G-1374) (one record, one row of the table). IQ and CGPA are the **features** (G-772), the input variables. LPA is the **target** (G-1949), the number we predict.

Student	IQ	CGPA	LPA
1	90	8	3
2	100	7	4
3	110	6	8
4	120	9	6
5	80	5	3

LPA is lakh rupees per annum. We build an ensemble of three models: model 1, then two decision trees. Real ensembles use many more; three are enough to see every step.

4. Stage 1: the mean

> **Key point:** For regression, the first model is not a tree at all: it is a single number, the mean of the target, predicted for every **observation** (G-1374).

The first model ignores the features. Whatever the IQ and CGPA, model 1 predicts the same number: the mean of the **target** (G-1949). In tree language model 1 is a single leaf.

1. **In words:** add up the target values and divide by the number of observations.

2. **Formula:**

$$F_0 = \frac{1}{n}\sum_{i=1}^{n} y_i$$

3. **Example:**

$$F_0 = \frac{3+4+8+6+3}{5} = \frac{24}{5} = 4.8$$

So model 1 says every student earns 4.8 LPA, even a student with CGPA 10 and IQ 120. We write F_0 for this first model, the **base prediction** (G-261), and store its answers in a new column, *pred1*: 4.8 in every observation. Why the mean is the right start is shown in step 1: the best constant is the mean.

5. Pseudo-residuals: the mistakes of the current model

> **Key point:** The mistake on each **observation** (G-1374) is actual minus predicted. Gradient boosting calls it the pseudo-residual.

To tell the next model about the mistakes, we need one number per observation that says how wrong we were. We use actual minus predicted, called the **pseudo-residual** (G-1589).

1. **In words:** for each observation, subtract the current prediction from the true **target** (G-1949) value.
2. **Formula:**

$$r_i = y_i - F_0$$

3. **Example:** student 1 earns 3 and was predicted 4.8:

$$r_1 = 3 - 4.8 = -1.8$$

Student	LPA	pred1	res1
1	3	4.8	−1.8
2	4	4.8	−0.8
3	8	4.8	3.2
4	6	4.8	1.2
5	3	4.8	−1.8

A negative residual means we predicted too much; a positive one, too little. A residual of 0 would mean a perfect prediction. The name “residual” comes from linear regression (see the best-fit line); why **gradient boosting** (G-859) adds "pseudo" is explained in pseudo-residuals are negative gradients.

6. Stage 2: a tree that predicts the mistakes

> **Key point:** Model 2 is a regression tree with the same features, but its **target** (G-1949) is the residual column, not the package. Model 2 learns how wrong model 1 is for each kind of student.

Training on the mistakes is the central idea of **gradient boosting** (G-859). Model 2 is a **regression tree** (G-1654; see **regression trees** (G-1654)):

- **features:** IQ and CGPA, as before;
- **target:** *res1*, the mistakes of model 1, not the package.

We are not asking tree 1 "what is this student's package?" but "how far off is model 1 for this student?". The better it predicts the mistakes, the better we can correct them.

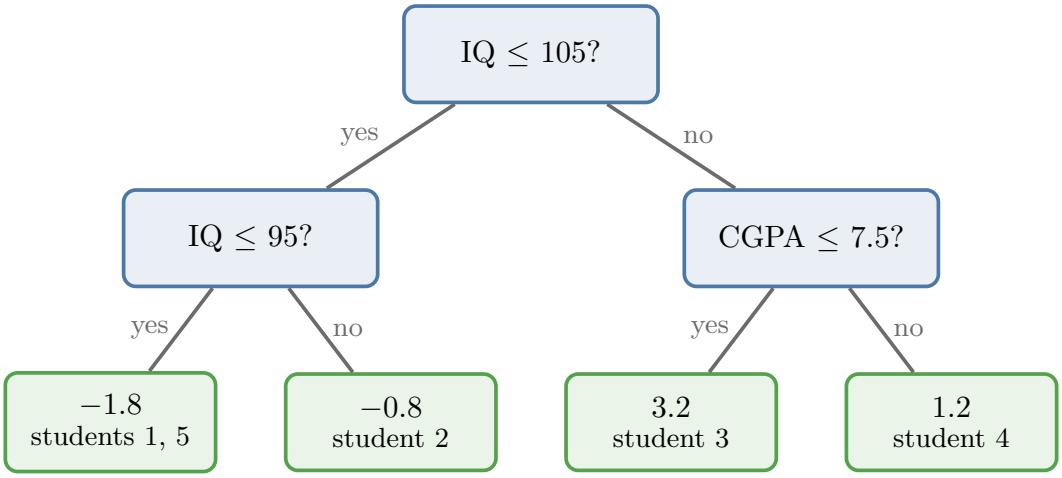


Figure 3: Tree 1, trained on the residuals: each leaf holds the residual of the students that reach it

Figure 3 shows the tree scikit-learn grows on this data. Student 1 (IQ 90) goes left at "IQ $\leq$ 105" and left again at "IQ $\leq$ 95", reaching the leaf -1.8: exactly student 1's residual. With only five **observations** (G-1374), every student gets a leaf of their own (students 1 and 5 share one, and their residuals are equal), so the tree's predictions *pred* on the training observations equal the residuals exactly.

Trees are the usual choice for these later models, and Friedman's original method is built around them (Friedman 2001), although any regression model could be used.

7. Adding the models together

> **Key point:** The ensemble's prediction is model 1's output plus model 2's output. Without anything else, this reproduces the training data exactly, which is overfitting.

To predict, we add the outputs of the models.

1. **In words:** start from the mean and add the tree's correction.
2. **Formula:**
$$F_1(x) = F_0 + h_1(x)$$
where h_1 is tree 1.
3. **Example:** student 1 (IQ 90, CGPA 8) gets 4.8 from model 1 and -1.8 from tree 1:
$$F_1 = 4.8 + (-1.8) = 3.0$$
which is exactly the package of 3 LPA.

Student 2 gets the prediction below, again exact:

$$4.8 - 0.8 = 4$$

Every training **observation** (G-1374) is now predicted perfectly, and that is a warning sign: the model has memorised these five students. A new student, not in the data, will probably be predicted badly. Memorising the training data is **overfitting** (G-1429; see overfitting and bias-variance).

8. The learning rate: small steps in the right direction

> **Key point:** We add only a fraction of each tree, the learning rate (typically 0.1). Each step is smaller, but it still points the right way; many small steps overfit less than one big jump.

The fix is to add only part of each tree's correction. That part is the **learning rate** (G-1067) η (eta), a number between 0 and 1, typically 0.1. With no learning rate, the multiplier is 1.

1. **In words:** multiply the tree's output by the learning rate before adding it.
2. **Formula:**
$$F_1(x) = F_0 + \eta\, h_1(x)$$
3. **Example:** student 1 with $\eta = 0.1$:
$$F_1 = 4.8 + 0.1 \times (-1.8)$$
$$F_1 = 4.8 - 0.18 = 4.62$$

The answer, 4.62, is still far from 3, but it moved the right way. More trees will bring it closer, one small step at a time. The same idea, shrinking every model's contribution so that

learning slows down and overfitting slows down even more, was called **shrinkage** (G-1796) in what the learning rate does.

Figure 4 compares the two choices for all five students. Watch the red bars: they land exactly on every actual package, which is memorising. The blue bars all move a tenth of the way from the mean 4.8 towards their **targets** (G-1949).

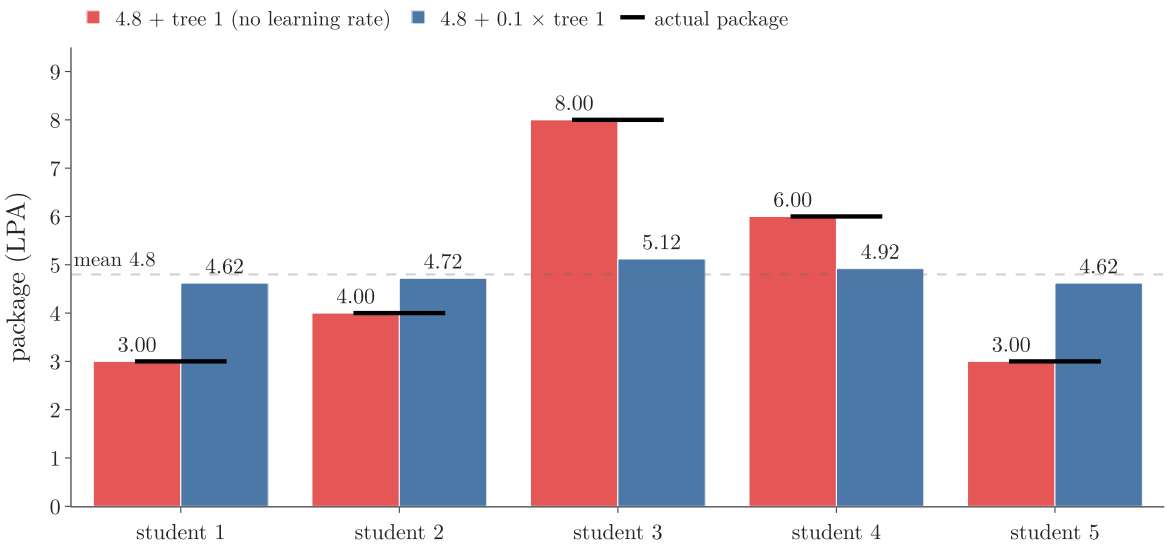


Figure 4: Five students after the mean plus tree 1: added in full (red), every prediction equals the actual package; added with learning rate 0.1 (blue), every prediction moves one tenth of the way from 4.8

The learning rate is the same for every tree after the first model: with $\eta = 0.1$, tree 1, tree 2 and every later tree are all multiplied by 0.1.

9. Stage 3: the residuals of the combined model

> **Key point:** The next tree is trained on the residuals of everything so far; with each stage the residuals move towards 0.

Before adding model 3, we measure how wrong the combined model of stage 2 is.

1. **In words:** actual minus the prediction of model 1 plus the scaled tree 1.
2. **Formula:**
$$r_i^{(2)} = y_i - \Big(F_0 + \eta\, h_1(x_i)\Big)$$
3. **Example:** student 1:
$$r_1^{(2)} = 3 - \Big(4.8 + 0.1 \times (-1.8)\Big)$$
$$r_1^{(2)} = 3 - 4.62 = -1.62$$

Student	LPA	res1	pred2	res2	pred3	res3
1	3	−1.8	4.62	−1.62	4.458	−1.458
2	4	−0.8	4.72	−0.72	4.648	−0.648
3	8	3.2	5.12	2.88	5.408	2.592
4	6	1.2	4.92	1.08	5.028	0.972
5	3	−1.8	4.62	−1.62	4.458	−1.458

Every residual moved towards 0: −1.8 became −1.62, 3.2 became 2.88. A residual of 0 means no gap between actual and predicted, so the goal is to keep adding models until the residuals are close to 0.

Figure 5 keeps going past the table. Watch the residual bars on the right: each tree removes a tenth of what is left, so they shrink by the same share at every stage. After 50 trees every residual is within 0.02 of 0, and the blue bars fill the outlines of the actual packages.

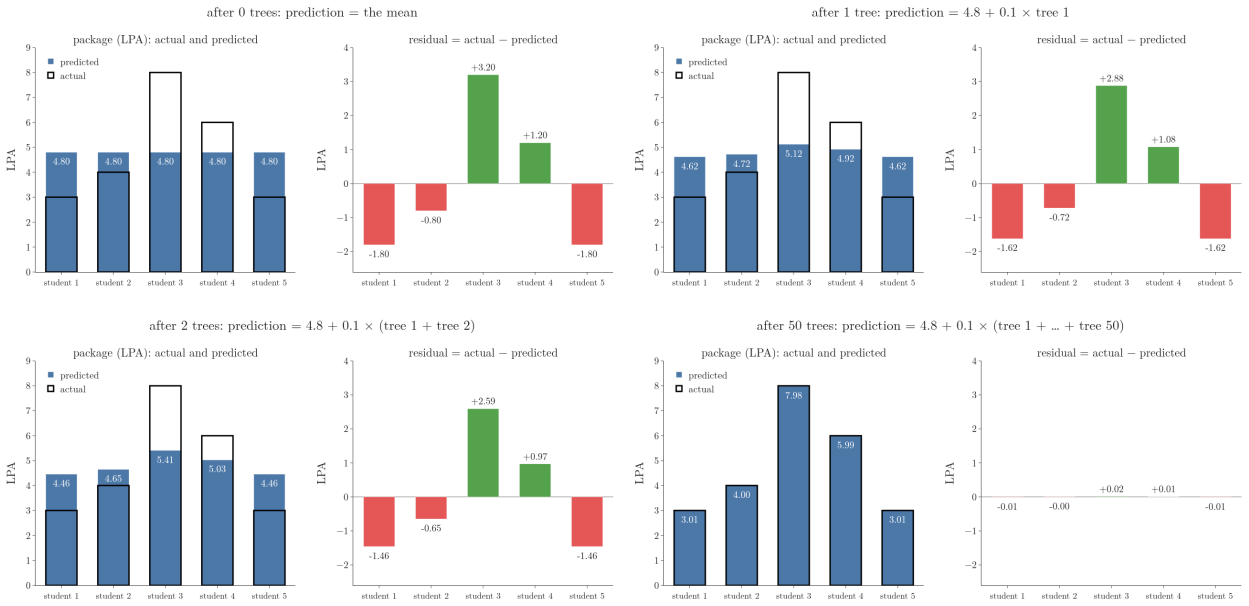


Figure 5: Gradient boosting on the five students, learning rate 0.1. Left: actual package (outline) and the model's prediction (blue bar). Right: the residuals the next tree is trained on

Model 3 is again a **regression tree** (G-1654): features IQ and CGPA, target *res2*. Model 3 predicts the mistakes of models 1 and 2 together. On this tiny dataset model 3 splits like tree 1, with leaves −1.62, −0.72, 2.88 and 1.08. Adding 0.1 times its output gives *pred3*, and *res3* is smaller again.

10. Predicting for a new student

Key point: Send the new observation through every tree, multiply each tree's output by the **learning rate** (G-1067), and add everything to the mean.

1. **In words:** the mean, plus the learning rate times each tree's output.
2. **Formula:**
$$\hat{y} = F_0 + \eta\, h_1(x) + \eta\, h_2(x) + \cdots + \eta\, h_M(x)$$
where M is the number of trees.
3. **Example:** a new student with IQ 60 and CGPA 4.9 lands in the leaf "IQ $\leq$ 95" of both trees, with outputs -1.8 and -1.62:
$$\hat{y} = 4.8 + 0.1 \times (-1.8) + 0.1 \times (-1.62)$$
$$\hat{y} = 4.8 - 0.18 - 0.162 = 4.458$$

So the model predicts about 4.46 LPA. With five **observations** (G-1374) and two trees the answer is rough; with real data and hundreds of trees, the same procedure gives very good predictions.

Figure 6 builds the sum step by step; watch each tree pull the answer down a little from the mean.

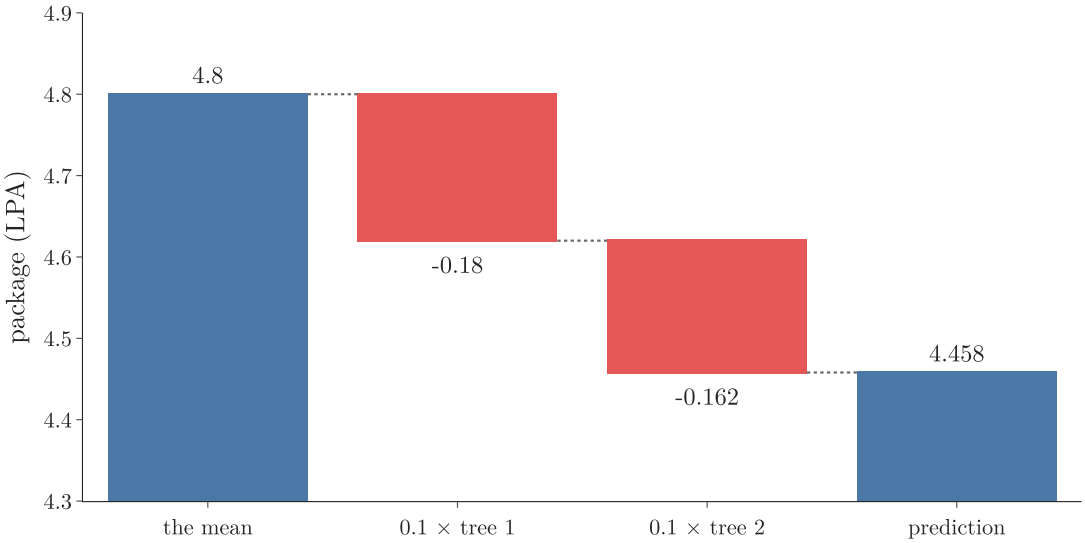


Figure 6: The prediction for the new student (IQ 60, CGPA 4.9): the mean 4.8, then 0.1 times each tree's output, -0.18 and -0.162, giving 4.458. The axis starts at 4.3 so the small steps show

11. Watching it fit a curve

> **Key point:** On 100 points along a curve, the mean is a flat line; each tree bends the model towards the data, and after many trees the model starts chasing the noise.

The same steps on a bigger dataset show what each stage does. We take 100 points with one **feature** (G-772) x between -0.5 and 0.5 and the **target** (G-1949) $y = 3x^2$ plus a little random noise (normal, with standard deviation 0.05): a U-shaped, non-linear relationship. Each tree has at most 8 leaves.

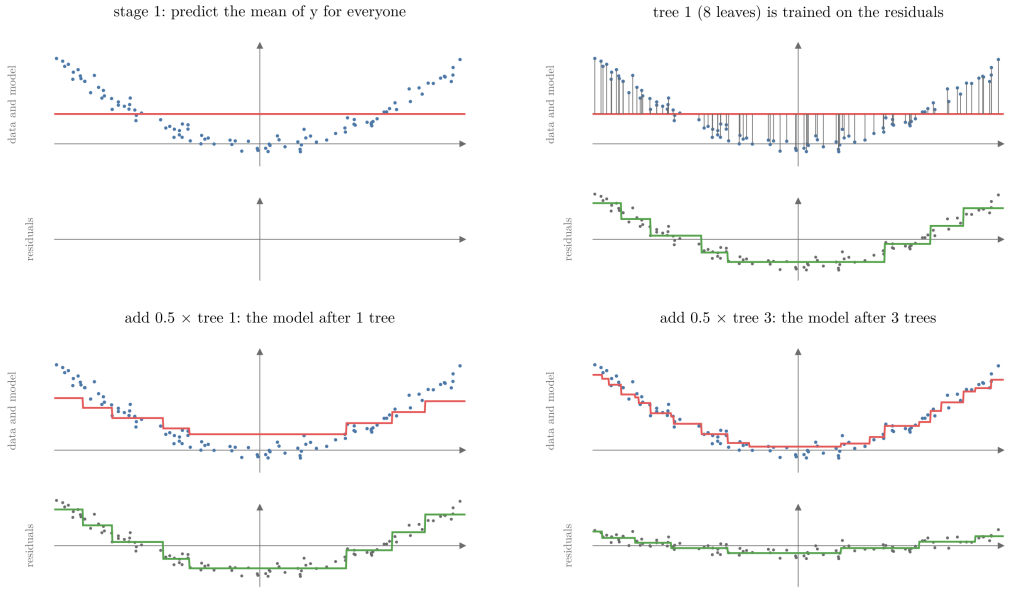


Figure 7: Gradient boosting on the curve, learning rate 0.5: residuals of the current model (grey), the tree trained on them (green), and the model after adding half of it (red)

Figure 7 runs three stages, with **learning rate** (G-1067) 0.5:

1. **Stage 1:** the model is the mean of y, 0.265: a flat red line.
2. **Residuals:** the grey sticks are the residuals; the lower panel shows them as points.
3. **Tree:** a tree with 8 leaves fits those residuals: the green steps.
4. **Update:** half of the green steps is added to the red line, which bends towards the data. The residuals shrink, and the next tree has less to fix.

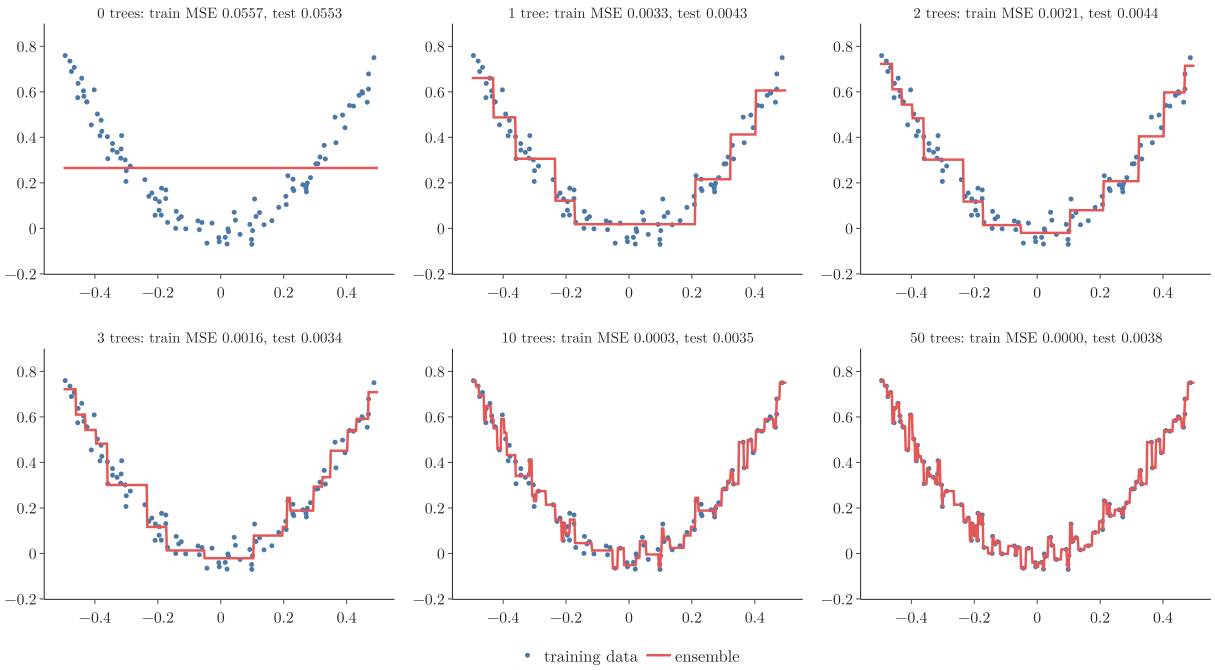


Figure 8: The model after 0, 1, 2, 3, 10 and 50 trees (learning rate 1, 8 leaves per tree), with training and test error

Figure 8 continues further, with learning rate 1. One tree already captures the U shape, and the training error falls from 0.0557 to 0.0033. After 10 trees the red line already zigzags between the training points (training error 0.0003), and after 50 trees it passes through every one of them (training error 0). The test error rises: averaged over 20 fresh datasets of the same kind, each tested on 5,000 new points, from 0.00450 after 3 trees to 0.00506 after 50 (Figure 8's titles show the single dataset drawn). Rising test error with falling training error is overfitting again, this time from too many trees.

Python: The whole loop is a few lines.

```
from sklearn.tree import DecisionTreeRegressor

pred = np.full(len(y), y.mean())          # stage 1: the mean
trees = []
for _ in range(n_trees):
    residual = y - pred                   # actual minus predicted
    tree = DecisionTreeRegressor(max_leaf_nodes=8)
    tree.fit(X, residual)                 # target: the residuals
    pred = pred + lr * tree.predict(X)    # small step
    trees.append(tree)
```

Each tree is limited to 8 leaves by `max_leaf_nodes=8`. scikit-learn's `GradientBoostingRegressor(loss="squared_error", learning_rate=lr, max_leaf_nodes=8, max_depth=None)` gives the same predictions to 15 decimals; the

Notebook checks this. (`max_depth=None` matters: its default `max_depth=3` would limit the trees as well.)

12. Learning rate against number of trees

Key point: As in AdaBoost, a smaller **learning rate** (G-1067) needs more trees but reaches a lower test error and overfits more slowly.

The trade-off is the one of the trade-off with n_estimators, so the two are tuned together. On the curve, with 8 leaves per tree, averaged over 20 fresh datasets with 5,000 test points each:

Learning rate	Lowest test error	Reached after	Test error after 200 trees
1.0	0.00448	4 trees	0.00506
0.5	0.00364	6 trees	0.00506
0.1	**0.00350**	35 trees	0.00463

With learning rate 1 the training error reaches 0 after about 50 trees, while the test error climbs back up. Learning rate 0.1 needs more trees, but its best test error is the lowest of the three, and after 200 trees it has climbed the least. ESL §10.12.1 reports the same: smaller learning rates give better test error and need more trees. (The noise in y alone gives a test error of $0.05^2 = 0.0025$, the floor no model can beat.)

Figure 9 runs the three learning rates side by side. Watch the stars in the lower panel: learning rate 1 reaches its lowest test error first (4 trees) but the highest; learning rate 0.1 takes 35 trees to get there and lands lowest, and its curve climbs back the slowest. In the upper panels, learning rates 1 and 0.5 zigzag through the noise after 50 trees, while 0.1 still follows the U shape with far fewer jumps.

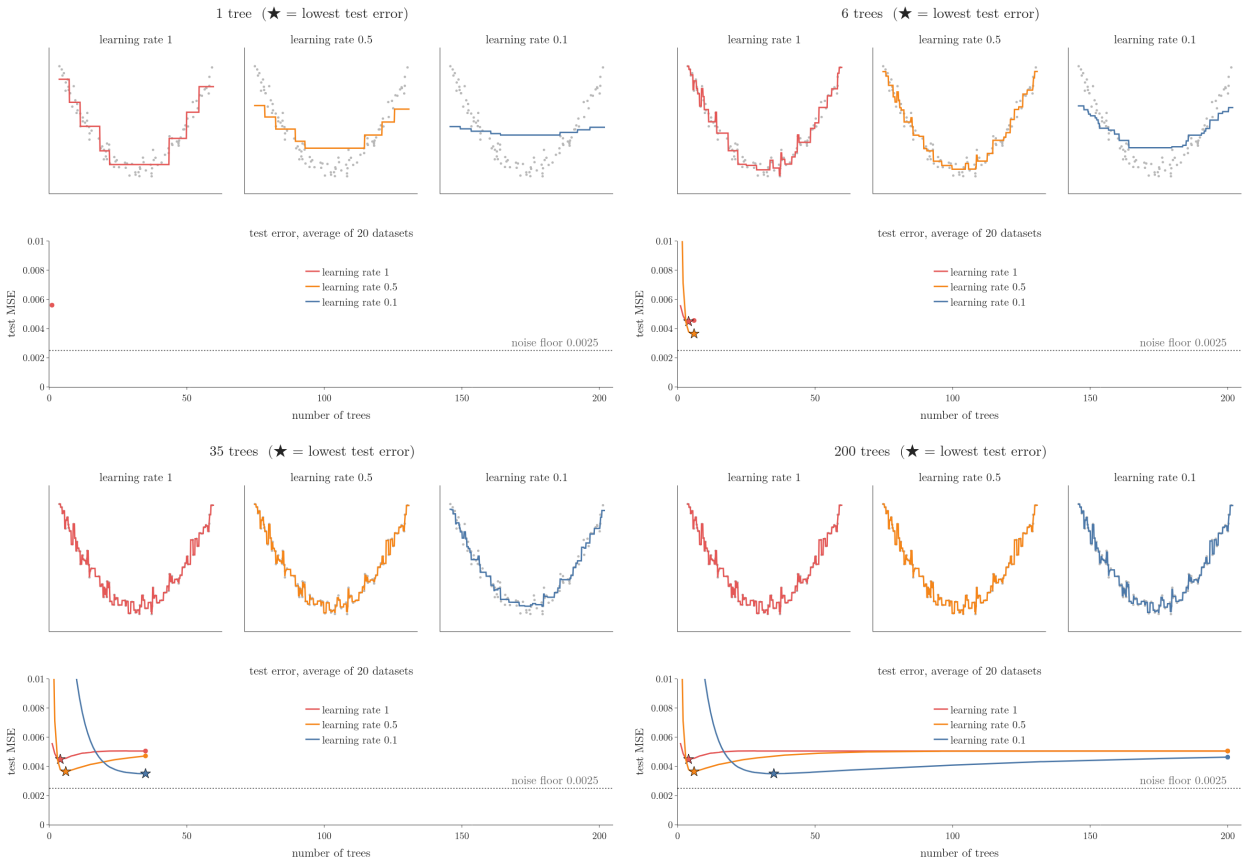


Figure 9: Gradient boosting with learning rates 1, 0.5 and 0.1 (8 leaves per tree), tree by tree. Top: the model on the curve data. Bottom: test error averaged over 20 fresh datasets; stars mark each learning rate's lowest test error; the dotted line is the noise floor

The Notebook's playground (`app.py`, a Dash app) lets us change the number of trees, the learning rate and the leaves per tree, and watch the curve and both errors change.

13. Gradient boosting compared with AdaBoost

> **Key point:** Gradient boosting uses bigger trees than AdaBoost's stumps, and one **learning rate** (G-1067) for every tree instead of a separate say for each model.

	AdaBoost	Gradient boosting
Base models	decision stumps (depth 1, 2 leaves)	trees with more leaves, commonly 8 to 32
How mistakes are passed on	larger weights for misclassified observations	the next tree's target is the residuals
How models are combined	each model gets its own say α	every tree is multiplied by the same learning rate

	AdaBoost	Gradient boosting
First model	a stump	the mean of the target (regression)

Figure 10 shows why stumps are too small, on the first residuals of the curve data. Watch the left panel: one split cannot follow the U shape, while the 8-leaf tree on the right removes most of the error in one stage (0.0033 left, against 0.0348).

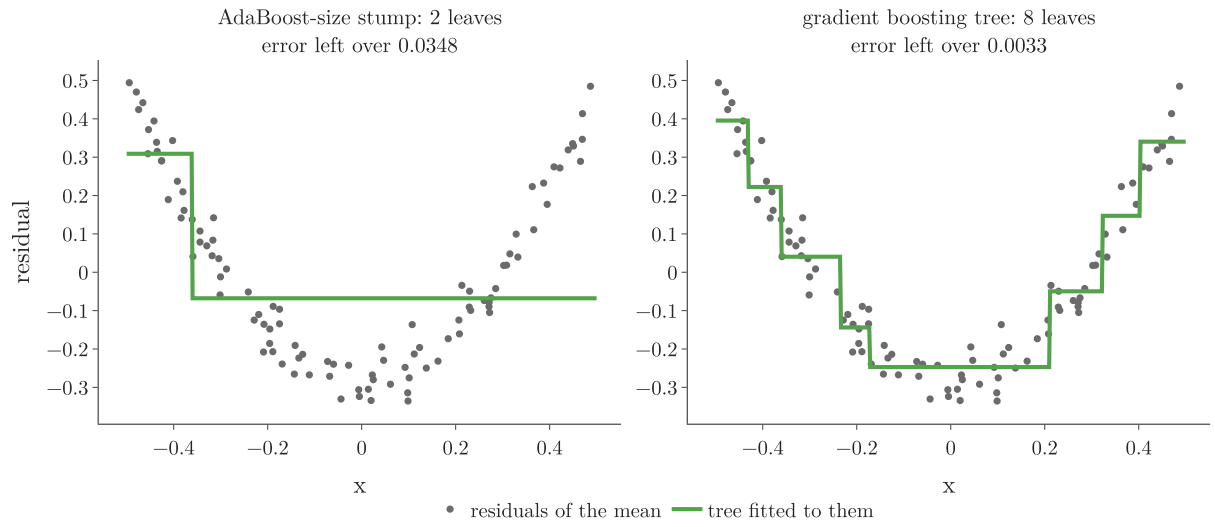


Figure 10: The residuals of the mean on the curve data, fitted by a stump with 2 leaves (left) and by a tree with 8 leaves (right); titles give the squared error left over

The leaf count depends on the data: about 8 leaves for small datasets, up to about 32 for big ones. Stumps are rarely enough: a tree with 2 leaves cannot capture any interaction between **features** (G-772), so boosted stumps suit only data where each feature acts on its own (ESL §10.11).

> **Extra:** Trees with 4 to 8 leaves work well in most cases (ESL §10.11); scikit-learn's default, `max_depth=3`, allows at most 8. Like the learning rate, the tree size is a hyperparameter, best set by cross-validation.

14. Summary

Step	What we do	Toy example	Why
Stage 1	predict the mean of y for everyone	4.8	the best single guess before any tree (why the mean)
Residuals	actual minus predicted	-1.8, -0.8, 3.2, 1.2, -1.8	one number per student saying how wrong the model is, and in which direction
Tree	regression tree, features x, target the residuals	leaves -1.8, -0.8, 3.2, 1.2	it learns how wrong the model is for each kind of student
Update	add learning rate $\times$ tree	$4.8 + 0.1 \times (-1.8) = 4.62$	a small step the right way; adding the whole tree memorises the data
Repeat	new residuals, new tree	$-1.8 \to -1.62 \to -1.458$	each stage moves the residuals towards 0
Predict	mean + $\eta\times$ (sum of tree outputs)	new student: 4.458	a new student gets every tree's correction, scaled as in training

- Gradient boosting is a **stage-wise additive model** (G-1867): each new tree predicts the mistakes of the ensemble so far, so each tree only has to learn what is still wrong.
- For regression the first model is the mean of the target, and the mistakes are the **pseudo-residuals** (G-1589), actual minus predicted, so a negative residual means the model predicted too much and a positive one too little.
- Adding whole trees overfits; the **learning rate** (G-1067) adds only a fraction of each, and many small steps generalise better.
- Too many trees overfit as well, so the number of trees and the learning rate are tuned together.
- Unlike **AdaBoost** (G-167): bigger trees (commonly 8 to 32 leaves), because a stump's single split cannot follow a curve or capture any interaction between features; and the same learning rate for every tree.
- So **gradient boosting** (G-859) answers its task in one loop: start from the mean, then keep adding a small part of a tree trained on the current mistakes.

15. Sources

Built from

- CampusX, "Gradient Boosting Explained | How Gradient Boosting Works?", YouTube, https://www.youtube.com/watch?v=fbKz7N92mhQ
- StatQuest with Josh Starmer, "Gradient Boost Part 1 (of 4): Regression Main Ideas", YouTube, https://www.youtube.com/watch?v=3CC4N4z3GJc (the opening comparison with AdaBoost, section 2)

Other references

- Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD 2016*, Section 1. arxiv.org/abs/1603.02754
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232. (Gradient boosting with regression trees, "TreeBoost".)
- Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS 2022, Datasets and Benchmarks Track*. arxiv.org/abs/2207.08815
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning* (ESL), 2nd ed., Springer. §10.11 "Right-sized trees for boosting" (4 to 8 leaves; stumps allow no interactions); §10.12.1 "Shrinkage".

16. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Gradient boosting (G-859)	A way to build a model from many trees added one by one: it starts from a simple guess, and each new tree is trained on the mistakes (pseudo-residuals) of the ensemble so far (a boosting algorithm).
Learning rate (η) in gradient boosting (G-1067)	The fraction of each tree's output that is added to the model, the same for every tree; typically 0.1.
Base prediction (F_0) (G-261)	The first model of gradient boosting; for regression, the mean of the target.
Pseudo-residual (G-1589)	The mistake on one row that the next tree learns; for squared error it is actual minus predicted.
Regression tree (G-1654)	A decision tree whose leaves predict numbers: the mean output of their training rows.

Term	Meaning
AdaBoost (Adaptive Boosting) (G-167)	A boosting algorithm that trains simple models (weak learners) one after another on reweighted data, and combines them by a vote in which each model counts by its alpha weight.
Additive modelling (G-174)	Building a complex function as a sum of simple functions, each capturing part of what the others missed.
Observation (G-1374)	One record of a dataset: one row of the data table.
Stage-wise additive model (G-1867)	A model built as a sum of base models added one at a time, one per stage; boosting builds this kind of model, each new part learning from the mistakes of the ones before.
Decision stump (G-559)	A decision tree with maximum depth 1: one question, two regions. Too weak alone, it is AdaBoost's usual weak learner.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Shrinkage (G-1796)	The pulling of a model's coefficients towards 0 by a penalty, as when Ridge's alpha grows; it makes the model rely less on any one feature and can reduce overfitting.